Autonomous Row Navigation for the farm-ng Amiga

Improvements, changes and upgrades — a complete record of the project from the first ROS 2 workspace to autonomous onion-bed straddling

Period covered	9 April 2026 – 16 October 2026
Repository	auto_navigation_vidalia (branch main, head 211858f)
Change history	272 commits
Robot	farm-ng Amiga · Jetson Xavier NX brain · Velodyne VLP-16 Puck Hi-Res · 2× OAK-D
Fields	Vidalia soybean (centre-residue strip) · Vidalia onion (72 in raised beds, 4 rows)
Test suite	293 passed, 2 skipped (hardware-free, numpy-only)
Report date	20 October 2026

How to read this report. Section 1 is a two-page summary. Section 2 is the timeline. Section 3 shows how the system fits together today. Sections 4–15 go subsystem by subsystem; each improvement is written as **Problem → Change → Result**, with the field evidence that motivated it. Section 16 lists field results, 17 the testing, 18 what is still open. Appendix A lists the command-line flags added along the way; Appendix B is the full commit log.

Contents

1. Executive summary **3**

Headline outcomes . . . 3

Key numbers . . . 3

2. Timeline **5**

3. The system today **6**

Field presets in one line . . . 6

4. LiDAR sensor, driver and calibration **7**

5. Row perception — soybean (centre-residue strip) **9**

6. Row perception — onion raised beds **11**

7. Cameras (OAK-D) and sensor fusion **13**

8. State machine and row-end logic **14**

9. Headland U-turn and serpentine coverage **14**

Safety nets around the turn . . . 15

10. Steering and speed control **16**

11. Safety **19**

12. Mapping (SLAM) and field coverage **19**

13. Experiment recording, metrics and visualisation **19**

14. Diagnostic and analysis tools **20**

15. Platform, connectivity and developer experience **20**

16. Field results **21**

17. Testing **21**

18. Open items and recommended next steps **21**

Appendix A — Command-line flags added (row_follow.py) **23**

Appendix B — Complete commit log (oldest first) **24**

1. Executive summary

Over six months the project went from an empty ROS 2 workspace to a field-tested, fully native-Python autonomy stack that follows crop rows on the Amiga with no map and no GPS. It straddles the residue strip in soybean and the raised bed in onion. It detects the end of the row, turns into the next row, records every run for publication, and offers three steering controllers: pure pursuit, MPC and reinforcement learning. Nearly every change was driven by a specific field failure, diagnosed from telemetry or raw scans, fixed at its root cause, and locked in with a regression test.

Headline outcomes

- **Field-proven row following.** v0.1.0 (23 Jun) was the first build to follow a soybean row autonomously in the field. On 15–16 Oct the onion stack tracked raised beds for 150–194 m per row with ~4–6 cm lateral standard deviation and 1.4–2.1° heading standard deviation, using all three controllers.
- **Months-old calibration mystery solved.** The LiDAR is a VLP-16 Puck Hi-Res (±10°), not a standard VLP-16 (±15°). The wrong angle table had turned a true 15°/0.80 m mount into a phantom 22°/1.17 m. After the fix, all 16 channels agree to ±0.1°.
- **Onion raised-bed mode, built from scratch.** It fits the 4-row comb, steers by the bed's soil relief when the seedlings are invisible to the LiDAR, detects the bed end, and never follows the grass.
- **Robust row-end and U-turn logic.** The turn design went through six generations: pivots, then an arc, then perception-closed, then IMU-measured, then a wheel-odometry fallback, then alignment-gated aborts. Added along the way: an APPROACH leg, post-turn settling, an off-field guard, creep-through-gap, and flicker-proof row-end counters.
- **Advanced controllers.** Pure pursuit gained integral action. MPPI MPC has a disturbance observer. The RL steering policies run in numpy only, with a drift integrator, slew bound and effort penalty. The simulator was system-identified from field telemetry.
- **Smooth driving.** False bed-end stops were eliminated. Speed no longer surges with perception confidence, and forward speed is rate-limited. Replayed on the 16 Oct runs, stops while following dropped from 382/163/57 to 18/15/1.
- **Publication-grade data.** `--record` writes a reproducible run folder: manifest, per-scan telemetry, metrics, raw scan time series, perception figures, a SLAM coverage map, and a POV video tool.
- **Engineering quality.** The test suite grew from 0 to 293 hardware-free tests. There are a dozen diagnostic tools, and the documentation (CLAUDE.md, CHANGELOG) is kept in step with the code.

Key numbers

Metric	Value	Context
Onion bed tracking, 15 Oct (bare bed)	lateral std 5.7 cm, heading std 2.0°	150 m in BED-SHAPE mode, relief 7.4 ± 0.7 cm, confidence 1.0
Onion bed tracking, 16 Oct (3 controllers)	lateral std 4.3–5.1 cm, heading std 1.38–2.07°	MPC 194 m, pursuit 128 m, RL 71 m
LiDAR channel agreement after Hi-Res fix	14.6° ± 0.1° (all 16 channels)	was a 9–19° fan
Pursuit + integral (sim, 200 episodes)	RMSE 23.5 → 19.2 cm, success 85.5 → 95.3 %	disturbance-cancelling baseline
MPC vs pursuit (sim)	RMSE 22.5 → 15.7 cm, success 87 → 96 %	steepest slopes: 0 % → 100 %
RL drift integrator (sim)	RMSE 21.4 → 8.5 cm, success 92 → 99.5 %	steep slopes: pursuit 7 %, RL 93 %
RL field-effort retrain	|ω| 0.147 → 0.090 rad/s on real scans	6.7× pursuit → MPC level

Metric	Value	Context
Onion RL policy (real-estimate replay)	3.1× pursuit effort (was 6.4×)	sim xtrack 8.0 vs 9.7 cm
False bed-end gate trips, 16 Oct runs	108 / 46 / 22 → 0	replayed on logged contrast
Unit tests	0 → 89 (v0.1.0) → 293	all run without hardware

2. Timeline

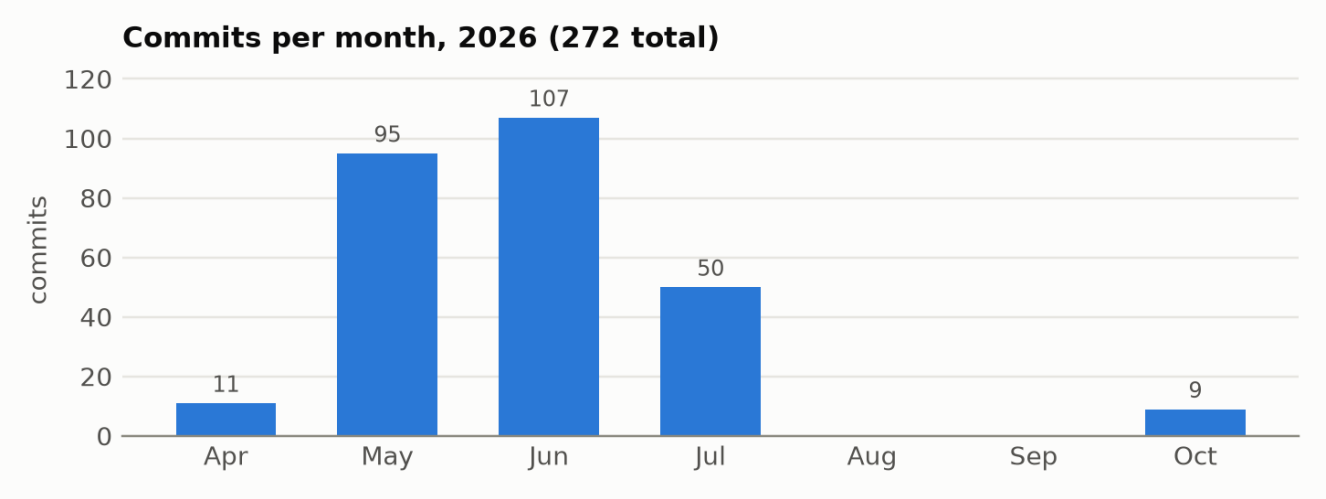


Figure 1 — Commit activity by month. Work paused in Aug–Sep between the soybean and onion seasons.

Phase	Dates	Theme	Main deliverables
1. Foundations	9–13 Apr	Workspace	ROS 2 Foxy package (vidalia_bringup: RTAB-Map, Nav2, Amiga bridge nodes, URDF, RViz); native-Python stack started for the brain's overlay filesystem
2. Bring-up	15–22 May	LiDAR, SLAM, first autonomy	VLP-16 driver + validation + Foxglove viewer; native 2-D SLAM (ICP, ray-casting, loop closure, wheel-odom fusion, deskew); LiDAR centre-row follower; safety zones; OAK-D integration (EventClient), camera-only stack, depth-edge detector
3. Fusion & soybean	29 May – 12 Jun	Tilt, ROS 2 bridge, soybean	Tilt correction; Docker ROS 2 bridge; EKF fusion; soybean dual-row mode; dual-camera IPM tracker; EKF sign fix; point-level fusion; cluster-centred PCA
4. Calibration & v0.1.0	22–23 Jun	Field baseline	Yaw correction, yaw→tilt order, terrain-adaptive crop band, canbus localhost fix — first field-validated build (tag v0.1.0)
5. Row-to-row	23–30 Jun	U-turn, data, controllers	Six generations of the headland turn, APPROACH, escapes; telemetry, --record, SLAM coverage, 3-D map; RL and MPC controllers; perception figures
6. Robustness	1–9 Jul	Root causes	Strip-lock; Hi-Res sensor fix; integral pursuit; residual RL; dense-canopy arbiter; creep-through-gap; heading ceiling; RL effort retrain; POV video
7. Onion	1–16 Oct	Raised beds	4-row bed mode; bed-relief steering; bed-end gate; onion sim-ID + RL/MPC retune; smooth driving; serpentine side planning

3. The system today

The Amiga brain's root filesystem reverts to the base image on every reboot. Only the NVMe home directory and Docker storage persist. That constraint shaped the architecture: the production autonomy stack is native Python running in the farm-ng venv, with no ROS install. ROS 2 lives in a Docker container that is used purely for live visualisation. ROS 2 Nav2/RTAB-Map remains available as a dev-PC reference stack.

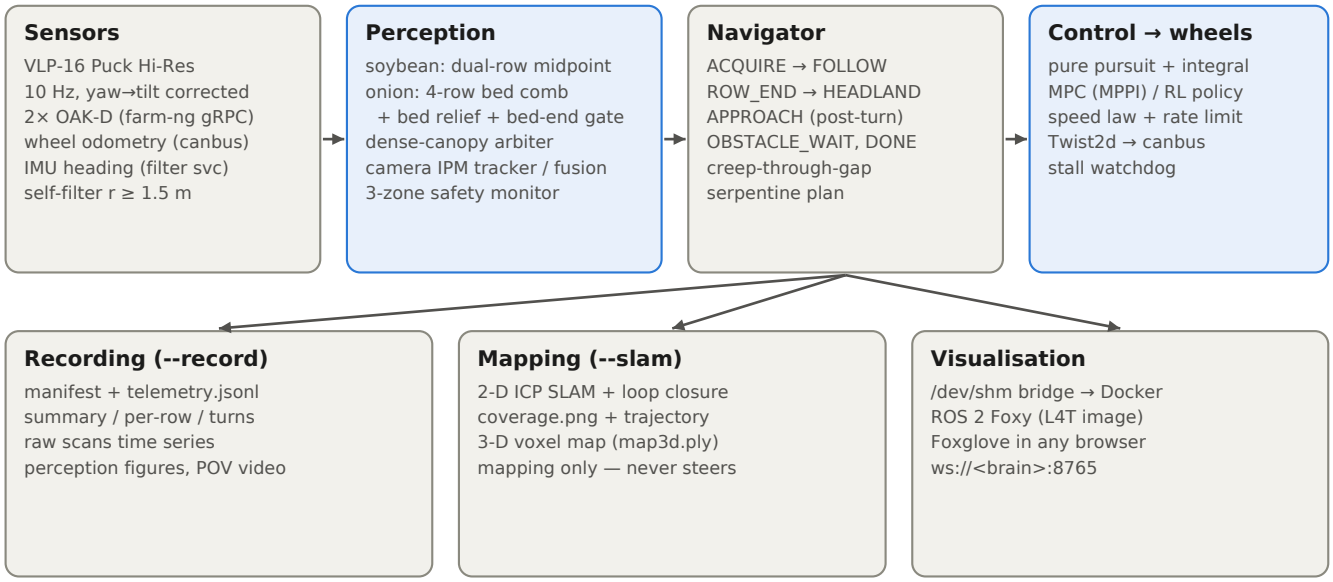


Figure 2 — Current data flow of the native LiDAR stack (scripts/row_follow.py). Mapping, recording and visualisation are side outputs; none of them can affect control.

Stack	Entry point	Use
Native Python — LiDAR (primary)	`scripts/row_follow.py`	Soybean and onion autonomy on the brain
Native Python — camera-only	`scripts/cam_follow.py`	No LiDAR / lightweight deployment
Native SLAM mapper	`scripts/slam_mapper.py`	Field map and coverage; mapping only
ROS 2 Foxy Docker bridge	`bash ros2_bridge/start.sh`	Live topics in Foxglove from any browser
ROS 2 Nav2 / RTAB-Map	`vidalia_bringup launch files`	Dev-PC reference (map-based navigation)

Field presets in one line

Field	Command (autonomous, recorded)
Soybean, one row	`python3 scripts/row_follow.py --auto --dual-row --record`
Soybean, serpentine	`python3 scripts/row_follow.py --auto --dual-row --rows 6 --headland --record`
Onion, one bed	`python3 scripts/row_follow.py --auto --preset onion --controller pursuit --record`
Onion, serpentine	`python3 scripts/row_follow.py --auto --preset onion --beds-left 4 --beds-right 0 --record`

4. LiDAR sensor, driver and calibration

The most important single fix of the project. For months the measured tilt and mount height disagreed with the tape measure and the phone level. The cause was not the mount; it was the driver's vertical-angle table.

Sensor is a VLP-16 Puck Hi-Res, not a standard VLP-16 (4–5 Jul)

Problem	The driver used the standard Puck's ±15°/2.00° channel table. The packet product-ID byte is 0x24, which is the Puck Hi-Res (±10°/1.33°). The wider angles stretched the reconstructed geometry, so a true 15° nose-down / 0.80 m mount looked like ~22° / 1.17 m. The tilt sweep, box test and ring fit all agreed on the wrong values.
Change	Replaced `VLP16_VERTICAL_ANGLES` with the Hi-Res table and re-derived the calibration: tilt 21.7° → 15.0°, yaw 0°, mount height 0.80 m (the floor now reads h ≈ 0, with no −0.37 m offset). The fix was propagated to SLAM, the ROS bridge TF, the launch files, the figure tools and the docs.
Result	Per-channel pitch collapsed from a 9–19° fan to 14.6° ± 0.1°. The raw-geometry fit gives 14.8° / 0.81 m, matching the tape and the level.
Where	`lidar/lidar_driver.py`, `tests/test_lidar_driver.py`

New diagnostics that cracked it

Problem	Every earlier calibration tool ran through the correction code itself, so they all inherited the same error.
Change	`diag_sensor_model.py` reads the product ID from the packet. `diag_pitch_rings.py` solves pitch AND height from raw ring geometry with no correction code in the loop: with correct angles all channels agree, with wrong angles they fan out. `diag_filter_imu.py` probes the IMU sources for pitch and roll.
Result	Any future mismatch between “pitch/height” and the tape can be diagnosed in one run.

Tilt correction (29 May)

Problem	The VLP-16 is mounted nose-down. Without correction, soybeans at h = 0.15 m read as 0.68 m (stuck in ACQUIRE forever), and ground 3 m ahead read as a 0.80 m obstacle.
Change	`tilt_correct_pts()` rotates the cloud about the robot's left–right axis. It is exposed as `--lidar-tilt`.
Result	Crop sits inside its height band and ground no longer triggers obstacles.

Yaw correction and correction order (23 Jun)

Problem	The earlier side-mounted sensor pointed 71° CCW of robot-forward. Tilt was also applied BEFORE yaw. The two rotations do not commute at that yaw, which left a ground ramp over 1 m that biased the heading and drove the robot off the row.
Change	Added `--lidar-yaw` and fixed the order to **yaw first, then tilt** (regression-locked). `diag_alignment.py` and the `diag_birdseye.py` forward-object locator measure yaw to about 1°.
Result	The yaw was re-measured after mount disturbances (71° → 66° after lens cleaning, → 0° for the July forward-facing re-mount). In the onion field, `--locate-range` (9 Oct) measured yaw 1.7° on a 6 m target, and the tilt sweep gave a flattest 12.2–12.3°; those values were passed on the command line for the October runs.

Driver performance and robustness (May)

Problem	Python packet parsing could not keep up with 10 Hz, scans fragmented, the robot frame showed up as obstacles, and a socket was not reusable after a crash.
Change	Vectorised numpy parsing (`scan_stream_np`), whole-revolution scan assembly, self-filter radius 1.0 → 1.2 → 1.5 m (clears the robot frame at ~0.72 m and the gantry), SO_REUSEADDR/SO_REUSEPORT, a pre-motion LiDAR health check, and a bird's-eye debug mode.
Result	A sustained 10 Hz stream with no self-detections.

Scan-stall watchdog (10 Jun)

Problem	If the LiDAR died mid-row, the loop blocked inside its `async for` and the robot kept driving on its last command.
Change	If no scan arrives for 0.5 s, the navigator actively commands zero velocity until the stream recovers.
Result	A dead sensor can no longer leave the robot moving.

Mount geometry kept in sync

Problem	Mount values were scattered across launch files, the TF, figures and the safety zones.
Change	Measured values were propagated everywhere: height 0.699 → 0.75 → 0.80 m, forward offset 0.85 m (1 ft ahead of the front tires), wheel track 75.5 in (tire zones ±0.959 m), and camera mounts 0.92 m high / 0.88 m lateral / 0.465 m behind the LiDAR.

5. Row perception — soybean (centre-residue strip)

The robot straddles a dark residue strip with a soybean row on each side (±0.38 m) and its wheels in the furrows. The perception target is the strip centre. The work below took the detector from a single PCA line fit to a self-checking, growth-stage-aware estimator.

Dual-row midpoint mode (4 Jun)

Problem	The original single-row (nearest-peak) detector locked onto one soybean row, not the strip between them.
Change	`--dual-row`: histogram the cross-row coordinate of crop-band points, pair the left/right peaks, and steer to their midpoint.
Result	The tracking target became the residue-strip centre.

Row-spacing prior and single-side fallback (10 Jun)

Problem	Weeds near the centreline formed a peak that won the pairing. When one row was occluded, the fallback returned the visible row itself as the target, so the robot steered onto the crop.
Change	Pick the left/right pair whose separation best matches the row spacing. A lone row is offset inward by half the spacing.
Result	Clutter no longer drags the midpoint, and a single visible row still gives the correct centre.

Cluster-centred PCA for heading (12 Jun)

Problem	The VLP-16 drops whole azimuth sectors (UDP loss), so one stripe is often shorter than the other. Whole-cloud PCA then tilts toward the line joining the stripe centroids. In the field this accumulated into a +12° heading bias and a rightward drift off the strip.
Change	Assign each point to its nearest peak, centre each cluster on its own centroid, and re-fit; iterate twice, with a 0.25 m gate and equal-weight clusters (which also removed a 4:1 density bias).
Result	Heading is unbiased regardless of stripe extents (`test_dual_row_truncated_stripe_*`).

Smoothing gates and decay (4 Jun – 12 Jun)

Problem	Sparse or round clusters made PCA heading flip ±50°. A one-scan dropout snapped the lateral by half a spacing. Empty scans made confidence collapse within 2 scans.
Change	A 30° heading outlier gate, a 0.30 m lateral outlier gate, a sparse-scan heading-quality gate, and an empty-scan decay of 0.75 (was 0.5).
Result	Stable estimates through VLP-16 empty scans and row ends.

Strip-lock continuity prior (1 Jul, strengthened 3 Jul)

Problem	A soybean field is periodic. “My strip, offset +half” and “next strip, offset −half” both match the spacing, so the pairing could alias onto the adjacent strip. The robot then crossed rows; RL, which reacts fastest, did it most often.
Change	Pair cost = |separation − spacing| + w·|midpoint − tracked lateral|, with w = 1.5, later 2.5 and exposed as `--strip-lock`. After a turn the prior is reset so re-acquisition prefers the centred strip.
Result	The detector stays on the strip it is following, for every controller.

Self-calibrating row spacing (27 Jun)

Problem	`--row-spacing` was a fixed geometry the operator had to know.
Change	It is now a seed. The detector refines it online from the observed peak separation, anchored to the seed after early versions wandered between 0.52 and 1.02 m. The live value is shown as sp=.
Result	Defaults work in most fields.

Terrain-adaptive crop band: grade, dip, roll (23 Jun, 4 Jul)

Problem	On slopes the crop ramped out of the fixed height band (a 5° grade shifts a 5 m return by 0.44 m). n collapsed from 450 to 46 and the robot stopped. Dips sank the canopy below the band. Cross-slopes made one row read higher than the other.
Change	Per-scan forward-grade detrend (20th percentile of z per range bin, slope only), a level shift onto the local ground in dips (only ever lowers the band), and a LiDAR-only lateral (roll) detrend. Shown live as `grade=`, `drop=` and `roll=`.
Result	Crop stays in the band on undulating fields; flat ground is unaffected.

Heading-runaway guards on grade (1 Jul – 9 Jul)

Problem	On grade spikes the PCA heading ran away (0 → −57°) while the lateral offset stayed ±0.05 m, which is geometrically impossible. The robot chased it into a ~75° turn. Later, at a sparse patch, a self-consistent but wrong fit reported +68…+86° together with −0.42 m lateral, and RL whipped the robot ~90° into the crop.
Change	A heading–lateral consistency cap, proportional to the offset (7° when centred, rising to 22° at 0.22 m; `--heading-cap`/`--heading-gate`). It was briefly disabled when it upset RL, then re-enabled once made proportional. On top of it, an **absolute heading ceiling** of 35° (`--heading-max`) that applies regardless of lateral offset.
Result	No controller can be driven into a violent turn by a non-physical estimate; normal following is untouched.

Dense / tall canopy: growth-stage-adaptive fit (6 Jul)

Problem	Once the canopy closed over (~0.75 m), the crop band filled solid. The cross-row histogram became a plateau, so the robot weaved ±0.1–0.29 m and ±5–22° and hopped rows despite high confidence.
Change	When ≥ 35 % of crop points are taller than 0.45 m, weight each point by its height above the strip floor, so the row ridges dominate and the strip reappears as a valley. `--no-dense-canopy`/`--canopy-tall-h`/`--dense-frac`.
Result	Early growth is byte-identical; dense canopy regains a clean strip.

Self-consistency arbiter + temporal trust filter (6 Jul)

Problem	A single 35 % threshold was brittle at the transition, and a single noisy scan jerked the steering.
Change	Fit BOTH experts (ground-density and canopy-height) and keep the one with the higher self-consistency (linearity × spacing match × agreement with the track). Scale the EMA gain by each scan's confidence.
Result	Emergent growth-stage handling; `mode` and `reliability` are logged and could seed a learned gate later.

Mid-row false headland in dense canopy (8 Jul)

Problem	The density fit's heading spiralled +8° → +61° over 1.5 m while reliability fell to ~0.1, mis-locating the flanking-row windows. Row-end read 1.0 and the robot took a U-turn mid-row with ~800 crop points in view.
Change	**Row-end density veto** (n ≥ 200 → never a row end; `--row-end-veto`): false ends had n ≥ 647, real ends n ≤ 78. **Reliability hold** (< 0.35 → hold the previous estimate; `--reliability-floor`).
Result	Fixed the mid-row turn and the dense weave at the source for pursuit, MPC and RL alike.

Row end on the longer line (3 Jul)

Problem	The row was declared ended when the first of the two soybean lines stopped.
Change	Row-end confidence uses the STRONGER flanking row.
Result	The robot follows whichever line runs longer to its true end.

Dense-vegetation signal tools (8 Jul)

Problem	Single scans in closed canopy are too noisy to fit reliably.
Change	`--accumulate N`: motion-compensated multi-scan accumulation for the row fit only (safety still uses the raw scan). `--lidar-intensity`: capture per-return intensity (NIR reflectance separates canopy from residue) into recorded scans for offline analysis.
Result	With `--accumulate 3`, real-scan heading peaks fell from 18° to 13°.

6. Row perception — onion raised beds

Vidalia onion layout: 72 in (1.83 m) raised beds, each carrying 4 rows at 11 in (0.279 m), plants 4 in apart. The robot straddles one bed with its wheels in the furrows. On 15 Oct the seedlings were pencil-thin and a few cm tall, so the VLP-16 could not see them, and the robot had to steer by the bed itself. Everything here is enabled by `--preset onion`.

4-row bed comb fit (1 Oct)

Problem	Single-row mode locked onto an inner onion row ~14 cm off the bed centre. The soybean 2-peak pairing aliased once the robot was ~9 cm off-centre.
Change	`find_bed_centre` fits the whole N-row comb. Each (peak, slot) proposes a centre, scored on misses, extras, residual, strip-lock and soil edges. Extras outweigh misses, so a planter skip never moves the lock.
Result	Capture range ~±0.35 m. Simulated bias ≤ 1 cm at 0 to ±35 cm start offsets, with skips, young rows, flat or 0.25 m beds, and yaw up to ~15°.

Crop band measured from the bed top (1 Oct, refined 15 Oct)

Problem	The 0.15 m bed soil sat inside the crop band, so a bare bed read as a confident row and the row end was never detected. In the field, bare soil read as a row at off = +0.8 m.
Change	The bed-top floor is the per-range dominant surface (1 cm histogram mode), fitted as a line with no slope dead-band. A bed centre beyond ±0.45 m (a wheel up on the bed) gets confidence 0.
Result	No more phantom rows on bare soil.

Row-aligned ROI and soil-edge tie-break (1 Oct)

Problem	At 0.28 m spacing a 5° yaw drifts the rows two spacings across the 7 m ROI, which biased the fixed box by −8 to −15 cm. A missing whole row at a start offset was ambiguous.
Change	Rotate the ROI box to the row direction (memoryless sharpness search, ±20°), and use the bed's soil edges as an independent centre.

Bare-bed RELIEF tracker (15 Oct)

Problem	The robot stopped at a “row end” mid-bed: seedlings invisible to the LiDAR meant no crop meant row end.
Change	Steer by the raised bed: build a cross-row soil-height profile with the forward ramp removed per range bin. The bed centre is the midpoint of the two shoulder edges at ±pitch/2; heading comes from the near and far halves. Confidence = relief ÷ 5 cm. Status `rel=6cm[BED-SHAPE]`.
Result	**Field, 15 Oct: 150 m in BED-SHAPE mode, lateral mean +0.2 cm / std 5.7 cm, heading std 2.0°, confidence 1.0 throughout.**

Bed END detection — never follow grass (15 Oct)

Problem	At the bed end the relief faded, the heading crept up to the 20° box cap, and the crop fit then followed GRASS for 6 m.
Change	**Bed-presence gate:** once the relief has been tracked, a smoothed relief below 0.6× its baseline means the bed has ended, with 0.8× hysteresis to resume. **Relief-based heading** (soil only, ±2°/scan rate limit, ±8° cap) replaces the box heading, which had ratcheted. **Crop lock:** a crop fit may steer only after ~2 s of agreeing with the bed AND with ≥ 60 % of its points on the 4 row lines (grass ≈ 30 %). The preset's headland exit is 3.5 m because the gate fires ~3 m before the end.
Result	On the real contrast trace: 0 false triggers over 150 m, end confirmed at 151.5 m. In simulation, grass is never re-locked. Status `[BED-END]`.

Relief dropout hold + coast (16 Oct)

Problem	Mid-row, the shape fit returned nothing for 2–4 scans (the bed floor jumped between soil and onion canopy): relief read 0.07 → exactly 0 → 0.07. The gate's EMA took the zeros as “bed ended”, 22–108 times per row, each a ~1 s stop.
Change	A missing fit is no evidence. Hold the gate and coast on the last bed geometry for up to 8 scans; a real bed end keeps reading low and still trips the gate.
Result	Replayed on all three 16 Oct contrast traces: 0 false triggers, with the real end still detected (≤ 0.1 m later on the 15 Oct trace).

7. Cameras (OAK-D) and sensor fusion

Camera driver over farm-ng gRPC (22 May)

Problem	The OAK-Ds sit on a PoE switch and `amiga_service` holds exclusive depthai handles, so direct depthai always failed with X_LINK_DEVICE_NOT_FOUND.
Change	Rewrote `oak_driver.py` to subscribe through EventClient on localhost:50010, decoding JPEG RGB and disparity. A missing oak1 now logs one warning and stops retrying instead of flooding the log.
Result	Cameras work on the brain alongside farm-ng services.

Camera-only autonomy stack (21–22 May)

Problem	There was no fallback when the LiDAR was unavailable.
Change	`cam_follow.py`: HSV green-centroid detector with a linearity-gated PCA heading (≥ 0.30, which stopped ±90° flips on round blobs), tunable `--hsv-*` flags, a colour-independent depth-edge detector (Canny/Hough + depth gap with local-maximum flank validation), and the same state machine.

Dual-camera ground-projection tracker (5 Jun → rewritten 10 Jun)

Problem	The original design (“each camera tracks its own side's row”) assumed inward-looking cameras. The cameras actually face forward and both see BOTH rows, so the old design was ambiguous and biased.
Change	Inverse perspective mapping: each green pixel is ray-cast onto the canopy plane to give metric robot-frame points, then fitted with the same pipeline as the LiDAR. Two-pass cluster-centred PCA removes the ~0.10 m / 2–3° per-camera bias, and a depth sanity gate rejects tall objects.
Result	The equal-weight two-camera mean cancels canopy-height bias to first order (the errors are equal and opposite).

Fusion levels and EKF (29 May – 11 Jun)

Problem	Camera and LiDAR were combined ad hoc. The EKF destabilised the loop and added lag.
Change	Phase-1 fusion (LiDAR + 3-D depth + EKF + wheel odometry). The EKF's predict signs were **flipped** for both lateral and heading and were corrected. `--cam-fusion point` pools camera ground points into one weighted row fit, with camera mass capped at 50 % of a full LiDAR scan, so a healthy LiDAR always wins.
Result	With a 0.35 m mis-calibrated camera the fused lateral moves < 0.05 m; camera points carry the fit through empty LiDAR scans and into the < 1.5 m blind zone.

Camera false obstacles and staleness (29 May – 12 Jun)

Problem	Single blocked frames and the robot's own gantry caused OBSTACLE_WAIT. A camera mounted at 0.92 m projected near-horizontal views to z ≈ 0.92 m, which read as “obstacle” in empty space. A dead stream froze fusion and blocked ROW_END forever.
Change	3-frame block debounce, horizon cropping, `--cam-near-filter`, `--no-cam-obstacles`, cam-depth-3d off by default, and a 1.5 s staleness gate.
Result	Safety is LiDAR-led; cameras can no longer wedge the robot.

8. State machine and row-end logic

States: ACQUIRE → FOLLOW → ROW_END → HEADLAND → (APPROACH) → FOLLOW, plus OBSTACLE_WAIT and DONE. The decision logic was moved into pure functions (`navigation/state_logic.py`) so each field failure could be reproduced as a unit test.

Field symptom	Root cause	Fix
FOLLOW ↔ ACQUIRE cycling	VLP-16 gives 1–3 empty ROI scans/s; confidence decayed past 0.35 in 2 scans	decay 0.75; `follow_miss_thresh` = 4 (stop but stay in FOLLOW); `acq_miss_thresh` = 2
Row end never reached the turn	the row-end and lost-lock signals raced	a real end needs a long CONTINUOUS absence (`row_end_frames` 15)
Spurious U-turn mid-row on a slope	4-scan trigger hit by crop flicker 0↔700	the miss counter resets the instant crop returns
Stuck in ACQUIRE at the real row end	residual clutter kept row-end confidence < 0.70	ACQUIRE row-end escape (only when entered from FOLLOW)
Escape still hung (16 s)	the counter reset on every row-end-confidence flicker	reset only on a confident re-acquire; hold otherwise
Escape hung 130+ scans	mid-range clutter (0.3–0.6) never counted	every scan that cannot re-lock counts toward the escape
Thrash ACQUIRE↔ROW_END↔ HEADLAND at thin patches	a stationary sensor cannot tell a thin patch from an end	**creep-through-gap**: creep up to 2 m (`--row-end-search`); the row reappears → FOLLOW, otherwise → row end
rows_done stuck at 0 for 10 min	every obstacle pause reset the row distance	only a new row resets it
ROW_END unreachable during an obstacle	OBSTACLE_WAIT blocked the row-end path	allow ROW_END from OBSTACLE_WAIT
Lurch at FOLLOW start	large initial heading error	optional `--align-heading` pivot (off by default)

9. Headland U-turn and serpentine coverage

Turning at the row end was the hardest problem in the project. Each generation fixed the previous one's field failure. Running underneath all of them is one fact: on a 4-wheel skid-steer, wheel odometry badly mis-reports rotation, so the turn must be closed on a better signal.

Gen.	Design	Field failure that retired it
1	Two in-place 90° pivots on wheel heading	pivots scrub all wheels; each “90°” was ~45°, so the total was ~90°
2	Pivots on the filter (IMU/GPS) heading, falling back to wheel	the filter only published with a GPS fix it did not have
3	Smooth arc (less scrub)	still closed on wheel heading; still ~90° physical
4	Perception-closed arc: arc until the next row is seen aligned	the arc under-rotated (slip widens the radius ~1.7×) and the detector locked onto grass at ~90°
5	IMU-measured arc: relative FilterState heading, even without convergence; perception may end the turn only after ≥ 150°/conf ≥ 0.72	in the wheel fallback the gates were wrong; with a live IMU it ended ~30° short
6 (now)	Gates 170°/178° in true IMU degrees; wheel fallback × `--turn-scrub-comp` 0.5; smoothstep ease-in; `rot=…[imu|wheel]` readout for one-run calibration	— (onion turn not yet field-validated)

Safety nets around the turn

- **APPROACH leg**: after the turn, creep forward (`--approach-speed`) until the next row locks; stop after `--approach-max-dist` if no row is found. Fixed “nothing in front, stuck in ACQUIRE”.
- **Post-turn settling**: for the first 2 m of the new row, a lost lock re-enters APPROACH instead of a stationary ACQUIRE.
- **Off-field guard**: `--post-turn-max-dist` (5 m) caps total post-turn travel before a stable FOLLOW, otherwise STOP.
- **Blind-spot confirmation**: the straight EXIT leg aborts back to FOLLOW if the row reappears, but only when it is ALIGNED and centred AND strong (conf ≥ 0.72). The earlier versions aborted on misaligned headland clutter (the robot never turned) and on sparse aligned clutter (endless thrash).
- **FilterHeading fix**: the IMU subscription was being cancelled after 5 s; fixed, so the live gyro is used.
- **Onion geometry**: radius 0.915 m (= half the bed pitch, so the robot lands on the adjacent bed); EXIT 3.5 m.

Serpentine side planning (16 Oct)

Problem	The first U-turn decides which side of the field gets covered. Starting on the second bed from the right (the rightmost bed was unplanted), the default RIGHT turn went toward the unplanted side, found no bed, and stopped. With the seedlings invisible to the LiDAR, it cannot tell a planted bed from an unplanted one.
Change	The operator states the layout: `--beds-left N --beds-right M`. `plan_serpentine` turns toward the side with more beds, sets rows = 1 + beds on that side (capping `--rows` at the field edge), and enables `--headland`. An explicit `--turn-dir` still wins.
Result	The robot can no longer run a U-turn off the field edge or into the wrong side.
Where	`navigation/coverage_plan.py`, `tests/test_coverage_plan.py`

10. Steering and speed control

All three controllers share one contract: they replace only the FOLLOW steering command. Each falls back to pure pursuit below the confidence threshold, is hard-clamped to `--max-angular`, and resets on every new row. The safety monitor, state machine and turn are untouched. All of it is numpy-only, so nothing new needs installing on the Jetson.

Pure pursuit tuning (3–4 Jun)

Problem	A 1.0 m look-ahead caused diverging oscillation when starting more than 15° off. The heading speed penalty starved the robot of steering authority (~0.10 m/s at a 22° entry).
Change	Look-ahead 2.0 m; speed divisor 1.0 rad; `--lookahead` and `--max-angular` flags.

Integral (disturbance-cancelling) action in the baseline (5 Jul)

Problem	Memoryless pure pursuit settles with a STANDING cross-track error on a cross-slope; this was the field slope-weave.
Change	A leaky (0.94), anti-windup (0.15 rad/s), confidence- and RATE-gated integral of the offset (k_i 0.15). It is the fallback under both MPC and RL.
Result	Simulation: RMSE 23.5 → 19.2 cm, success 85.5 → 95.3 %, no limit cycle.

MPC — MPPI with a disturbance observer (27 Jun)

Problem	Pure pursuit can only chase slope drift.
Change	`--controller mpc`: sample K steering sequences over an H-step horizon and apply the cost-weighted soft-argmin. A Luenberger drift observer and a slip observer feed the preview.
Result	Simulation: RMSE 22.5 → 15.7 cm, success 87 → 96 %; on the steepest slopes pursuit 0 % vs MPC 100 %.

Reinforcement-learning steering (27 Jun → 16 Oct)

Problem	Could a learned policy beat a geometric controller where it matters (drift, smoothness)?
Change	**Base:** numpy MLP [5→16→16→1] (385 params), trained with Evolution Strategies in a gym-style simulator with slip, drift, noise and dropout. An alive bonus prevents reward hacking. **Drift integrator** (5th input): RMSE 21.4 → 8.5 cm, steep slopes 7 % → 93 %. **Jerk sweep** (`--c-du` 0.3 → 8.0) halves jerk at negligible tracking cost. **Residual RL** learns a bounded delta on the pursuit baseline. **Slew bound** `--rl-slew` caps the change in ω per step. **Effort penalty** (`follow_field.npz`): on real scans |ω| fell 0.147 → 0.090 (from 6.7× pursuit to MPC's level) while keeping the best sim tracking (10.3 cm).
Result	RL went from “weird turning” to best-tracking and as smooth as MPC; it remains opt-in.

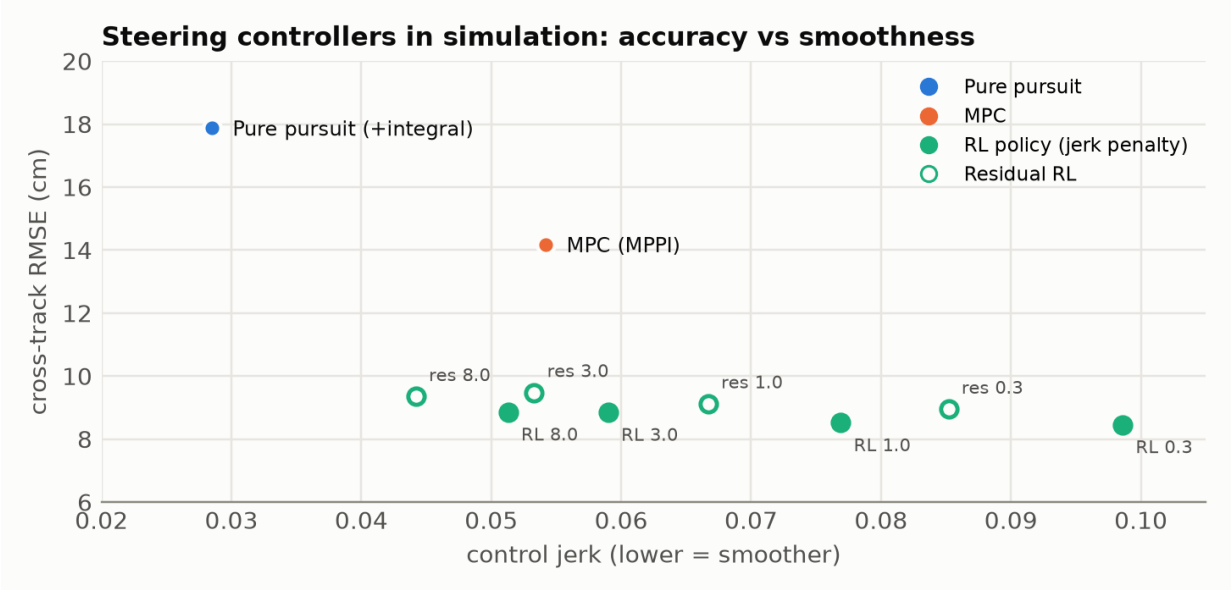


Figure 3 — Held-out simulation (200 episodes, identical disturbances). RL variants trade a little jerk for roughly half of pure pursuit's cross-track error; MPC sits between them. Source: results/controller_pareto.csv.

Onion-field sim system-ID, retrained policy, retuned MPC (16 Oct)

Problem	The soybean policy steered 6.4× pursuit's effort on real onion estimates. Default MPPI steered 5.3×, and raising its costs made it worse (12–19×).
Change	`calibrate_sim_from_run.py` fits white + AR(1) wander noise so that pursuit in the sim reproduces the field run's closed-loop statistics (within ~10 %). `follow_onion.npz` was trained on that fit; `replay_controllers.py` replays real estimates through every controller as an honest sim-to-real check. MPC calm setting: noise 0.20, temperature 0.3, observer 0.05.
Result	Onion RL: sim xtrack 8.0 vs 9.7 cm (pursuit), real-replay effort 3.1×; MPC 3.5×. Both are applied by `--preset onion`.

Smooth forward speed (16 Oct)

Problem	The operator saw sudden slow-downs and brake-like jerks. Speed was proportional to confidence, and the bed-relief mode (~0.9) and crop-fit mode (~0.7) switched about once a second, giving ±25 % speed steps on top of the false bed-end stops.
Change	`--speed-conf-full` (onion 0.65): full speed at any healthy lock, proportional below it. A forward-speed rate limiter (`--accel-limit 0.15`, `--decel-limit 0.5` m/s²) in FOLLOW/APPROACH; zero-speed and obstacle stops are never softened.
Result	Replayed on 16 Oct: zero-speed scans 382/163/57 → 18/15/1, speed CV 0.21–0.26 → 0.07–0.14. The robot now cruises near `--speed` (mean 0.15 → ~0.19 m/s at 0.20).

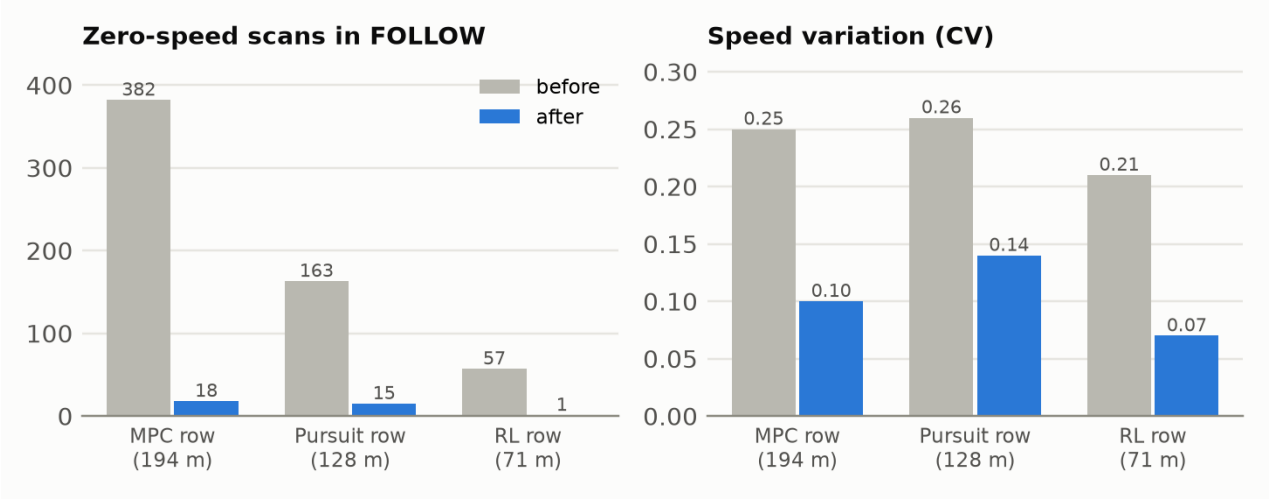


Figure 4 — Bed-gate dropout hold + speed law + rate limiter, replayed on the logged perception of the three 16 Oct rows. Steering is unchanged.

11. Safety

Element	Evolution	Current
Forward zone	permanent OBSTACLE_WAIT at 0.7/1.2/2.2 m from the robot frame and ground → self-radius + height fixes; width 0.97 m tripped on adjacent crop	2.5 m × ±0.60 m; obstacle height 0.50 m (soybean) / 0.75 m (onion)
Tire zones	adjacent canopy (0.80–0.84 m) and dried residue stalks (0.50–0.65 m) caused L-TIRE stops	±0.959 m track, ±0.25 m corridors; 0.65 m (soybean) / 0.85 m (onion)
Slope	tall crop on an up-slope read as an obstacle	forward-grade detrend in the height test (real obstacles still exceed it)
Camera obstacles	single-frame and gantry false positives	3-frame debounce, horizon crop, near filter; LiDAR-led
Status display	OBSTACLE_WAIT showed “safe=clear” when the camera was the cause	reason() always names the trigger
Loss of sensing	robot kept driving if the LiDAR stalled	0.5 s scan watchdog → zero velocity
Canbus	/twist routed via Tailscale and timed out; blocking sends	localhost services, retrying handshake, 0.5 s send timeout
Controllers	learned policy could misbehave	pursuit fallback, ω clamp, RL slew bound, absolute heading ceiling
Speed smoothing	—	rate limit never delays a stop

12. Mapping (SLAM) and field coverage

Native 2-D LiDAR SLAM (18–19 May)

Problem	There was no ROS on the brain, and a map was needed for the reference stack and for coverage.
Change	Point-to-point ICP with a sliding-window reference, ray-cast occupancy (log-odds), voxel downsampling, a constant-velocity model, trimmed ICP with outlier removal, velocity caps, scan-to-map loop closure, wheel-odometry warm start (canbus in its own thread), motion deskew, and a tuned ICP reject threshold (0.25 → 0.08 m, trusting odometry over a bad ICP).

SLAM yaw/tilt correction (26 Jun)

Problem	SLAM sliced the RAW cloud. Flat ground 6 m ahead read h ≈ 1.4 m inside the slice band, which smeared the map.
Change	`correct_scan` applies yaw → tilt before slicing; the band is [0.05, 1.50] m (`--slice-min 0.20` for structure only).
Result	Clean maps; regression-locked.

Coverage map, 3-D map, live mapping (26–27 Jun)

Problem	A single scan cannot tell whether the field was covered.
Change	`coverage.png` stamps the 1.92 m swath along the drift-corrected path; gaps are missed rows; it reports area, path length and redundancy. `--map-3d` registers full scans on the 2-D pose into a voxel map (`map3d.ply`). `row_follow.py --slam` maps while driving, in a decoupled thread.
Result	This is the controller A/B figure for the paper.

13. Experiment recording, metrics and visualisation

Capability	What it gives you
`--telemetry`	One JSON line per scan: state, confidence, n, lateral/heading, row-end, grade, command, safety zones, turn state, spacing, bed relief/floor/mode

Capability	What it gives you
`--record`	runs/run_<controller>_<ts>/: manifest (git commit, args, calibration), telemetry, summary.json, metrics_flat.csv, per_row.csv, turns.csv, coverage/trajectory/map
`run_metrics.py`	Cross-track RMSE/MAE/p95/max, heading RMSE, effort, jerk, speed, state budget, events, per-row and per-turn outcomes
`analyze_run.py`	Re-compute tables offline; `--compare` builds the field A/B CSV across runs
Raw scan time series	Background recorder (never blocks driving): first scan + every 10th, with index.csv; `--save-scans`
Perception figures	Auto-rendered a few seconds into each run from a real scan: 3-D, bird's-eye scope, annotated (PNG + SVG); old-matplotlib safe
`render_pov_video.py`	Cinematic POV video from a run: chase / pov / high / orbit / scope / split views, HUD, GIF + MP4
ROS 2 bridge	/dev/shm IPC → Docker (dustynv L4T Foxy) → Foxglove WebSocket; went through RViz/noVNC and rosbridge on the way; systemd autostart

14. Diagnostic and analysis tools

Tool	Purpose
`diag_birdseye.py`	Bird's-eye view, `--tilt-sweep` pitch calibration, ROI height profile, forward-object yaw locator, `--locate-range`, `--mount-height`
`diag_pitch_rings.py`	Pitch and height from raw ring geometry (independent of the pipeline)
`diag_sensor_model.py`	Velodyne product ID from the packet
`diag_filter_imu.py`	Probe filter/OAK IMU for pitch, roll and heading
`diag_alignment.py`	Interactive LiDAR yaw alignment
`diag_row_fit.py`	Per-scan dual-row fit diagnostics + raw scan capture
`replay_scans.py`	Offline A/B of perception settings on recorded scans (accumulation, intensity)
`replay_controllers.py`	Real estimates through every controller (sim-to-real check)
`calibrate_sim_from_run.py`	System-identify the RL simulator from a field run
`train_rl.py` / `eval_controller.py` / `gen_controller_pareto.py`	Training, held-out evaluation, Pareto table
test_lidar_*.py, test_canbus.py	Step-by-step bring-up checks (scan validate, live view, slow drive)

15. Platform, connectivity and developer experience

- **Brain constraints respected**: everything lives in the NVMe home and the farm-ng venv; Python 3.8 dataclass compatibility; farm-ng venv auto-added to sys.path; depthai pinned to 2.22.
- **Canbus**: AmigaTpdo1 decoding fixed (decode=False), subscription in its own thread and loop, localhost-first host selection, retrying handshake, non-blocking send with timeout.
- **Clean shutdown**: gRPC teardown errors, double-Ctrl+C tracebacks and asyncio noise suppressed; the robot is always commanded to stop on exit.
- **Startup UX**: LiDAR health check, camera presence check, banner with full safety/speed/turn configuration, presets (`--preset soybean|onion`), one-line serpentine plan printout.
- **Documentation**: CLAUDE.md (hardware facts, every flag, a known-issues table with root causes), CHANGELOG, README; a standing rule to commit and push each completed change; `setup_claude_code.sh` for the brain.

16. Field results

Date	Field / run	Result
21 May	Warehouse cardboard row	First autonomous LiDAR row following (tuned for the 6 ft tire track)
23 Jun	Soybean — v0.1.0	First field-validated row follow: strip centred, stable heading, survives real terrain
late Jun	Soybean — U-turn trials	Successive turn failures (90° under-rotation, grass lock, short turn) drove generations 1–6
early Jul	Soybean, grown/dense	Mid-row false headland, weave and RL whip diagnosed from telemetry and raw scans; fixed at source
early Jul	Soybean RL/MPC/pursuit	Run completed 3/3 rows and 160 m with clumsy transitions → creep-through-gap
15 Oct	Onion, bare bed	150 m in BED-SHAPE mode, lateral std 5.7 cm, heading std 2.0°, conf 1.0; bed end found, then grass followed → gate
16 Oct	Onion — MPC	194 m, lateral std 5.1 cm, heading std 1.38°, mean |ω| 0.018, jerk 0.0109
16 Oct	Onion — pure pursuit	128 m at 0.30 m/s max, lateral std 4.3 cm, heading std 1.41°, |ω| 0.013, jerk 0.0011 (smoothest)
16 Oct	Onion — RL (follow_onion)	71 m, lateral std 4.9 cm, heading std 2.07°, |ω| 0.018, jerk 0.0033
16 Oct	Onion — first U-turn	Turned right (default) toward an unplanted bed, wheel-odometry rotation; found no bed in 3 m and stopped safely → serpentine plan

All three controllers detected the real bed end on 16 Oct. The remaining roughness (false stops, speed surges) is addressed by the 16 Oct changes and should be confirmed in the next field session.

17. Testing

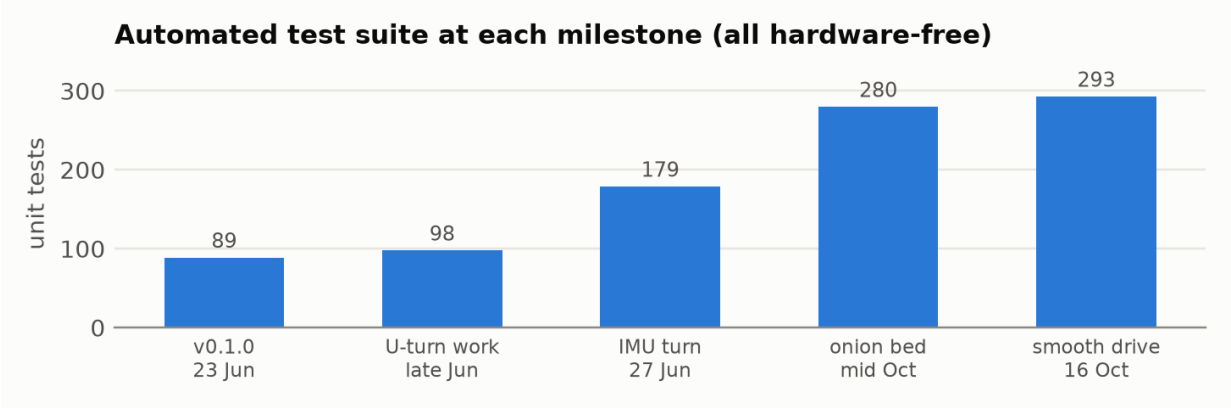


Figure 5 — Test-suite size at each milestone.

Every test runs on any machine with numpy and pytest; no LiDAR, cameras, canbus or farm-ng SDK is needed. Field failures are reproduced as tests before they are fixed: slope dropouts, flicker at the row end, misaligned headland clutter, truncated stripes, dense-canopy runaway, and the real recorded bed-relief contrast trace (`tests/data/field_20261015_onion_bed_contrast.npz`). Coverage spans perception, EKF, controllers (including closed-loop simulation), safety zones, turn and odometry, tilt and fusion, SLAM, coverage, metrics, recording, figures, RL, MPC, the onion bed and onion RL, and serpentine planning.

18. Open items and recommended next steps

- **Field-validate the onion serpentine.** Run `--beds-left/--beds-right` and watch `rot=`. The IMU heading was not live on 16 Oct (wheel fallback), so calibrate `--turn-scrub-comp` so that `rot` reads ~180 when the

robot has truly turned 180°.

- **Confirm smooth driving in the field.** Expect near-constant cruising at `--speed`; lower it if the old pace (~0.15 m/s) is preferred.
- **Live gyro for turns.** A FilterState heading without a GPS fix (or another IMU source) would remove the scrub-compensation guess entirely.
- **Auto side detection.** Once onions are tall enough to be seen, the LiDAR could check which side has planted beds; today the operator states it.
- **Learned perception gate.** The arbiter's mode/reliability telemetry from `--record` runs is a ready training set.
- **Intensity discriminator.** Analyse recorded Nx4 scans before wiring intensity into the live fit.
- **Paper A/B.** The same bed with `--controller pursuit|mpc|rl --record`, then `analyze_run.py --compare`.

Appendix A — Command-line flags added (row_follow.py)

Area	Flags
Calibration	`--lidar-yaw, --lidar-tilt, --self-radius, --roi-x`
Detection mode	`--dual-row, --bed-rows, --bed-pitch, --no-bed-shape, --preset soybean|onion, --row-spacing (self-calibrating seed), --strip-lock`
Terrain & canopy	`--no-ground-detrend, --heading-cap, --heading-gate, --heading-max, --no-dense-canopy, --canopy-tall-h, --dense-frac, --reliability-floor, --row-end-veto, --accumulate, --lidar-intensity`
State machine	`--acquire-conf, --row-end-min-dist, --row-end-search, --align-heading, --align-speed`
Headland / coverage	`--rows, --headland, --turn-dir, --beds-left, --beds-right, --headland-exit, --headland-speed, --headland-turn-rate, --headland-radius, --reacquire-conf, --turn-scrub-comp, --approach-speed, --approach-max-dist, --post-turn-max-dist`
Control	`--controller pursuit|mpc|rl, --policy, --rl-slew, --rl-residual, --residual-scale, --mpc-noise, --mpc-temp, --mpc-drift-gain, --lookahead, --max-angular, --speed, --speed-conf-full, --accel-limit, --decel-limit`
Safety	`--obstacle-height, --tire-height, --forward-dist, --forward-half-width`
Cameras	`--camera, --cam-fusion, --cam-left-id, --cam-right-id, --cam-x, --cam-height, --cam-y-fwd, --cam-yaw-deg, --cam-pitch-deg, --cam-stop-dist, --cam-block-frames, --cam-depth-3d, --cam-near-filter, --no-cam-obstacles, --ekf`
Data & mapping	`--telemetry, --record, --save-scans, --no-slam, --slam, --map-3d, --voxel-3d, --ros2-bridge, --debug, --save-dir, --no-validate`

Each flag is documented with its default and rationale in CLAUDE.md (“Full CLI Flag Reference”).

Appendix B — Complete commit log (oldest first)

#	Date	Commit	Summary
1	04-09	2c481a3	Migrate amiga_slam to vidalia_bringup for ROS2 Foxy on Ubuntu 20.04
2	04-10	c110431	Initialize repository with README for Vidalia Foxy SLAM workspace
3	04-10	04fd42e	Initialize repository for Vidalia autonomous navigation (ROS 2 Foxy, Ubuntu 20.04)
4	04-10	df66357	Initial commit: auto_navigation_vidalia for ROS2 Foxy
5	04-12	bc491ee	Migrate vidalia_bringup ROS2 Foxy SLAM package to main
6	04-12	cd44768	Add install_farmng.sh, save_map.sh, test_canbus.py scripts
7	04-12	0588d64	Add src/vidalia_bringup package files and Python nodes
8	04-12	5908e13	Add Python nodes: amiga_ros2_bridge, amiga_odometry, amiga_velocity_bridge
9	04-12	96c4a2c	Add all vidalia_bringup files: launch, config, nodes, rviz, urdf, tests
10	04-13	1904713	Add native Python stack and fix ROS2 references for Amiga brain
11	04-13	6a4d799	Add step-by-step test suite and fix remaining ROS2 script references
12	05-15	36b62ee	Fix Python 3.8 dataclass compat, add LiDAR network setup and run.sh
13	05-15	bfacd36	Add VLP-16 Foxglove WebSocket visualizer script
14	05-15	195126f	Fix visualizer: await asyncio.sleep(0) + wait_opened() to ensure port binds
15	05-18	62fd4b4	Update LiDAR mount transform and add slow-drive test
16	05-18	f2cf6c6	Add LiDAR validation and live visualization tools
17	05-18	2e4e7ed	Add LiDAR validation and live visualization tools
18	05-18	3a8699c	Add ground-relative obstacle filter and pre-motion LiDAR validation
19	05-18	e0a013d	Merge claude/update-lidar-mount-Z4yhh: ground-relative obstacle filter and LiDAR validation
20	05-18	71b9ecf	Sync all LiDAR mount values to measured x=0.959 m, z=0.699 m
21	05-18	961e1e7	Add native-Python 2D SLAM with live Foxglove Studio visualization
22	05-18	d5f6b0f	Fix Foxglove schema parse errors on all SLAM channels
23	05-18	1b43237	Fix FLOAT32 enum value and velodyne→map TF error in Foxglove publisher
24	05-18	0d6c9e8	Add no-Foxglove SLAM mapper with PNG map save
25	05-18	f51883b	Merge claude/update-lidar-mount-Z4yhh: no-Foxglove SLAM mapper and map I/O
26	05-18	ad0ec32	Fix Ctrl+C hang in slam_mapper: remove custom signal handler
27	05-18	25d79ce	Merge claude/update-lidar-mount-Z4yhh: fix Ctrl+C hang in slam_mapper
28	05-18	1189981	Add SO_REUSEADDR to LidarDriver socket
29	05-18	b775027	Merge claude/update-lidar-mount-Z4yhh: SO_REUSEADDR on LiDAR socket
30	05-18	5e89d8e	Add SO_REUSEPORT to LidarDriver socket alongside SO_REUSEADDR
31	05-18	0f3280f	Merge claude/update-lidar-mount-Z4yhh: add SO_REUSEPORT to LiDAR socket
32	05-18	9818f51	slam_mapper: print waiting message and first-packet confirmation
33	05-18	6697475	Merge claude/update-lidar-mount-Z4yhh: LiDAR startup diagnostic in slam_mapper
34	05-18	f21672b	Merge claude/build-real-map-68DyL: SLAM mapper, map save/load, PNG export, ICP improvements
35	05-18	73b3dc4	Fix SLAM map quality: ray casting, sliding-window reference, voxel downsample
36	05-18	c510bd1	Fix map_io: _L_CLAMP renamed to _L_MAX in OccupancyGrid
37	05-18	90e973a	Add constant-velocity motion model to SLAM for turn tracking
38	05-19	6cbc59b	Add outlier removal and trimmed ICP to clean up map noise
39	05-19	a8b387d	Add velocity caps and scan-to-map loop closure to SLAM
40	05-19	ad4ae90	Add wheel odometry fusion and tune loop closure for open fields
41	05-19	885e1cf	Fix canbus config loading so wheel odometry actually activates
42	05-19	4d5ec5f	Auto-add farm-ng venv to sys.path so canbus works without activating venv
43	05-19	105a334	Add canbus connection diagnostics: connecting message + 5s timeout warning
44	05-19	f14f3c5	Fix canbus: AmigaTpdo1 is in farm_ng.canbus.packet, requires decode=False
45	05-19	eb9aaa1	canbus: try localhost first, then Tailscale; 3s probe timeout per host
46	05-19	16d1406	canbus: remove broken asyncio.wait_for probe, subscribe directly
47	05-19	4715a9a	canbus: run gRPC subscription in its own thread+event loop
48	05-19	1662cf2	Add motion deskew + tune ICP for sparse agricultural fields
49	05-19	c7b6340	slam: lower ICP_REJECT_THRESHOLD 0.25 → 0.08 m to trust odom over bad ICP
50	05-20	476ad12	nav: add LiDAR center-row follower for mapless onion-field navigation
51	05-21	f0c6278	Add online LiDAR centre-row following autonomy (native stack, no map)
52	05-21	002e600	Tune row follower for 6 ft tire track and warehouse cardboard row
53	05-21	549feb9	Fix scan fragmentation, robot-frame self-detection; add LiDAR debug mode
54	05-21	eaa145b	Vectorize VLP-16 parsing to sustain 10 Hz; add bird's-eye debug view
55	05-21	49a355f	Raise obstacle threshold above crop band; halve obstacle-clear delay
56	05-21	a8adc1f	Increase self_radius 1.0 → 1.2 m to filter robot mounting crossbar
57	05-21	c6c6341	Raise obstacle_height above LiDAR mount height to clear robot frame
58	05-21	4d7d4e7	Raise self_radius to 1.5 m; add point counts to safety reason string
59	05-21	40c923e	Add --tire-height flag; print safety config in banner
60	05-21	6281ab3	Lower acquire_conf to 0.45; add --acquire-conf CLI flag
61	05-21	52f0179	Add OAK-D camera integration (depth obstacle + visual row detection)

#	Date	Commit	Summary
62	05-21	f6378fa	fix(oak_driver): use depthai 2.22.0-compatible resolutions
63	05-21	0845077	docs(CLAUDE.md): document row-follow stack progress, tuned parameters, and findings
64	05-21	e165ac2	feat: camera-only autonomous row following (no LiDAR)
65	05-21	584d478	fix(cam_follow): detect missing cameras at startup and abort with helpful message
66	05-22	4210bc4	Rewrite oak_driver.py to use farm-ng EventClient instead of depthai
67	05-22	90a6ce5	docs: confirm OakFrame image_data encoding from live inspection
68	05-22	49069db	Fix cam-only ACQUIRE never exiting due to EMA convergence boundary
69	05-22	5f3f6e4	Suppress gRPC teardown RuntimeError on event loop close
70	05-22	be99ade	Fix gRPC teardown errors: proper task cancellation before loop close
71	05-22	5488fdc	Ignore SIGINT during task cleanup to prevent double-Ctrl+C traceback
72	05-22	11b81ed	Add --hsv-h-lo/hi/s-lo/v-lo flags for tunable colour detection
73	05-22	04ff1f4	Add depth-edge row detector and --detector flag to cam_follow.py
74	05-22	445ef4a	Fix depth-edge lateral: require local maximum, fix spurious open-space offset
75	05-22	60587cf	Fix depth-edge heading bias and nanmean warning
76	05-22	a57c04c	Fix Ctrl+C traceback during gRPC cleanup
77	05-22	c6223bc	Fix PCA heading instability and obstacle strip geometry for side cameras
78	05-29	1f1f275	Add LiDAR tilt correction for 15° forward (nose-down) mount
79	05-29	f8a0b5d	Fix camera obstacle strip geometry in row_follow.py --camera mode
80	05-29	7eb1111	Fix oak1 gRPC spam, add camera block debounce, update docs
81	05-29	f14e0bd	Add Docker ROS2 Foxy visualization bridge for RViz2 / professor demos
82	05-29	4799c20	ROS2 bridge: Foxglove WebSocket + autostart — no laptop install needed
83	05-29	1cb8b36	Switch ROS2 bridge to L4T-optimised Foxy image for Jetson Xavier NX
84	05-29	d358d36	Comprehensive docs update: Amiga Brain config, ROS2 bridge, all flags
85	05-29	ae65668	Fix expired ROS 2 apt signing key in Dockerfile
86	05-29	c9655e8	Fix ROS 2 apt key: overwrite signed-by keyring, not legacy keyring
87	05-29	bc66cfd	Switch to rosbridge_suite; fix duplicate apt source from previous attempt
88	05-29	365bf32	Fix entrypoint: drop -u flag; drop redundant -p with --net=host
89	05-29	0933855	Switch ROS2 bridge to RViz2 + noVNC (no Foxglove)
90	05-29	f812637	Fix ROS2 setup.bash path for foxy-desktop image
91	05-29	871963a	Fix VNC port conflict and QoS mismatch
92	05-29	7a397a9	Wait for x11vnc to be ready before starting websockify
93	05-29	8f3956b	Replace nc -z with bash /dev/tcp for VNC readiness check
94	05-29	1bfb606	Redirect noVNC root / to vnc.html automatically
95	05-29	1d06e1a	Fix VNC 'Connecting...' hang: remove /dev/tcp readiness probe
96	05-29	ddff924	Remove --index-file flag (not supported by this websockify version)
97	05-29	2ce5880	feat: Phase 1 sensor fusion — LiDAR + OAK-D 3D depth + EKF + wheel odometry
98	05-31	58942ae	fix: prevent ACQUIRE→OBSTACLE_WAIT loop and FOLLOW confidence oscillation
99	05-31	b04b95e	fix: EKF-aware ACQUIRE — survives VLP-16 alternating empty/full scans
100	05-31	9c4d3ed	feat: complete RViz2 visualization — row centreline, state overlay, robot footprint
101	05-31	4fd4c2e	Add fused LiDAR+camera visualization and /camera_points topic
102	05-31	d039f02	Add three-view RViz visualization: LiDAR, OAK-D cameras, fused nav
103	05-31	0562f11	Fix Left/Right OAK-D display labels — swap topic subscriptions
104	05-31	59fe167	Fix oak0/oak1 camera side assignment — oak0 is physically right camera
105	05-31	25a1749	Improve sensor fusion: velocity prediction, innovation gate, confidence floor, debounce
106	05-31	5de47ce	Auto-scale depth_to_points intrinsics for non-default frame sizes
107	06-01	62fdbe8	Fix RViz Left/Right OAK-D topic subscriptions
108	06-03	aa53c75	Default camera obstacle detection to 3D height-filtered mode
109	06-03	21bcd07	Add --forward-dist/--forward-half-width flags; enable EKF by default
110	06-03	1448ced	Add --lookahead and --max-angular CLI flags; reduce default lookahead 2.0→1.0 m
111	06-03	da48f24	Add heading pre-align rotation in ACQUIRE to prevent lurching at FOLLOW start
112	06-03	2412738	Fix stuck-in-ACQUIRE, improve row-end robustness, and clarify ALIGN status
113	06-03	d761e83	Disable pre-align by default; narrow forward zone to 0.60 m
114	06-03	c56e379	Raise tire-zone height threshold to 0.92 m; sync navigator default
115	06-04	1d135d6	Fix camera 3D cloud triggering false OBSTACLE_WAIT from robot's own gantry
116	06-04	afa3ec4	Fix camera gantry false positives and EKF heading corruption
117	06-04	43bea71	Allow ROW_END from OBSTACLE_WAIT; lower row_end_min_dist to 1.5m
118	06-04	bacde3f	Fix camera extrinsics for forward-facing mount (was assuming side-inward)
119	06-04	a09cf20	Crop above-horizon rows from camera depth cloud to eliminate gantry false positives
120	06-04	f26a1bc	Use math.ceil for horizon-crop row boundary in depth_to_points
121	06-04	7e9a6f7	Reduce heading-error speed penalty in pure-pursuit controller
122	06-04	2013506	Apply measured robot geometry: LiDAR height, camera mount, safety zones
123	06-04	71ccf87	Update cam_x to 0.88 m (measured inter-camera span 1.76 m / 2)
124	06-04	36f8c53	Revert forward_half_width to 0.60 m — 0.97 m triggers on adjacent crop rows
125	06-04	27897c3	Fix diverging oscillation: lookahead 1.0→2.0 m, EKF off by default

#	Date	Commit	Summary
126	06-04	37bc9ad	Fix camera 3D cloud horizon false positives at FWD ~1.1 m
127	06-04	a2d03f7	Update camera mount geometry: height 1.000→0.920 m, y_fwd -0.505→-0.465 m
128	06-04	70f9dad	Disable cam-depth-3d by default — camera at 0.92 m causes false OBSTACLE_WAIT
129	06-04	5b11450	Fix FOLLOW→ACQUIRE cycling and heading oscillation on sparse scans
130	06-04	57ae714	Fix ACQUIRE counter cycling and heading gate deactivation on empty scans
131	06-04	cdb5e2c	Add soybean field support: dual-row centre-residue detection + updated defaults
132	06-05	89684e5	Soybean autonomy: dual-camera flanking-row tracking + closed-loop U-turns
133	06-05	737c0b4	Fix cam_follow.py hanging indefinitely on canbus verification
134	06-05	1429a69	Fix soybean cam-only: disable depth obstacle in dual-row mode, fix HSV defaults
135	06-05	61c5630	Lower acquire_green default 0.08→0.05 for sparse young soybean seedlings
136	06-05	a19d837	Widen default HSV range for young soybean seedlings in field conditions
137	06-05	5f3a51b	Fix soybean row tracker: restrict column search to inner half, fix depth
138	06-05	8b6c404	fix(cam_follow): canbus timeout is non-fatal — always attempt auto drive
139	06-05	3d51654	fix: add 0.5s timeout to canbus send_twist to unblock navigation loop
140	06-05	d3259c1	fix(cam_follow): increase camera warm-up wait from 2.5s to 5.0s
141	06-10	86a2020	Fix EKF motion-model signs, dual-row fallback geometry; add safety watchdogs and unit tests
142	06-10	ff3e73e	Rewrite dual-camera tracker for forward-facing stereo-baseline geometry (ground projection / IPM)
143	06-10	2505a6b	docs: standing rule — commit and push every completed change to main
144	06-11	9b8183b	Add opt-in point-level LiDAR+camera fusion (--cam-fusion point)
145	06-11	de05d5b	Add setup_claude_code.sh — persistent Claude Code CLI install for Amiga brain
146	06-11	b8377e4	Suppress gRPC asyncio cleanup noise on Ctrl+C
147	06-11	6819850	Fix default --lidar-tilt from 15.0 to 0.0 (field-verified flat mount)
148	06-11	d2573a6	Revert --lidar-tilt default to 15.0° (hardware-confirmed nose-down mount)
149	06-11	95c2d49	Fix --lidar-tilt default to 0.0: mount roll not pitch (debug-verified)
150	06-12	cb05f6a	Add --cam-near-filter to ignore robot-frame depth returns (default 0.50 m)
151	06-12	b1237bb	Add --no-cam-obstacles flag for soybean/dual-row fields
152	06-12	8e094fb	Raise soybean tire-zone height threshold 0.35→0.65 m; fix reason() display bug
153	06-12	ae73fdc	Wire --roi-x into the dual-camera tracker; document four-row field geometry
154	06-12	ea5abd4	Fix incorrect four-row geometry in CLAUDE.md; add alignment guidance
155	06-12	062e826	Improve precision and field-deployment UX for soybean/onion navigation
156	06-12	f4a173d	Apply cluster-centred PCA to all dual-row fits — fixes heading bias from VLP-16 sector dropouts
157	06-12	6d20323	Merge origin/main (precision/UX improvements) into cluster-PCA fix
158	06-12	d39cc31	Add sparse-scan heading quality gate to RowDetector._smooth()
159	06-12	37ac7e7	Equal-weight cluster normalisation in _cluster_centred_pca — fixes 4:1 density bias
160	06-12	f963c42	Add diag_row_fit.py — per-scan dual-row fit diagnostics + raw scan capture
161	06-22	360c759	Update confirmed hardware geometry: mount height 0.75m, 15° pitch, 75.5in track
162	06-22	a244087	Revert --lidar-tilt default to 0.0 — data verifies mount has roll not pitch
163	06-23	98ac51c	Add diag_alignment.py — interactive LiDAR yaw alignment diagnostic
164	06-23	edb2d3f	Fix diag_alignment.py: keep LiDAR driver open across all three steps
165	06-23	24aeed7	Fix diag_alignment.py hang between steps: async input + persistent stream
166	06-23	54654a9	Rewrite diag_alignment.py with background producer + queue
167	06-23	22bf30e	Add diagnostics to diag_alignment.py: producer error catch + timeouts
168	06-23	f24188a	Add --lidar-yaw correction: sensor Y+ is 71° CCW of robot-forward
169	06-23	bc3aed9	Add diag_birdseye.py and --lidar-yaw to diag_row_fit.py
170	06-23	b9f9324	Save birdseye.png to repo root by default; add to .gitignore
171	06-23	fe3b525	diag_birdseye: default yaw=71°, forward-only bird's-eye view
172	06-23	c9065a5	Fix LiDAR correction order: apply yaw before tilt
173	06-23	71dbbaa	Make tilt-sweep ground metric population-stable
174	06-23	b91e64f	Set field-calibrated LiDAR defaults: yaw 71, tilt 21.5
175	06-23	a45d5b0	Fix ROS2 bridge TF: cloud is pre-corrected, use identity rotation
176	06-23	591134f	Make canbus handshake robust (retry + longer timeout)
177	06-23	27c89fb	Point canbus/filter services at localhost (run on the brain)
178	06-23	cf67133	Terrain-adaptive crop band: detrend forward ground grade
179	06-23	608a20d	diag_birdseye: print height profile inside the detector ROI
180	06-23	3b593ab	Terrain dip: lower crop band onto local ground (level shift)
181	06-23	69eadfd	Add CHANGELOG documenting the v0.1.0 field-validated baseline
182	06-23	4bf2132	Headland turn: IMU/GPS filter heading for pivots + distinct shift distance
183	06-23	b51b4df	Fix row-end never reaching the headland turn (loss-vs-end race)
184	06-25	7225ae6	diag_birdseye: forward object locator for precise yaw verification
185	06-25	cadb0d5	Re-calibrate LiDAR yaw 71 -> 66 after mount disturbance
186	06-25	4779818	Fix spurious headland turn on slopes (row-end needs continuous absence)
187	06-25	7c12a95	Add APPROACH leg: drive into next row after the U-turn (fix stuck-after-turn)
188	06-25	ff39c82	Add ACQUIRE row-end escape: unstick the robot at the real row end
189	06-25	b2cf5ef	Keep creeping (not stall in ACQUIRE) while settling onto the next row after a U-turn

#	Date	Commit	Summary
190	06-26	18a765b	Arc U-turn (fixes ~90° skid-steer turn) + off-field guard + blind-spot row-end confirm
191	06-26	da9aba4	SLAM: yaw/tilt-correct the LiDAR cloud before slicing (fixes ground-flooded map)
192	06-26	e79856d	3-D field mapping (--map-3d): register full scans on the 2-D SLAM pose
193	06-26	165e9a1	Perception-closed U-turn: arc until the next row is seen aligned ahead
194	06-26	3c9dbf6	IMU-measured U-turn: gate completion on real rotation, reject grass at ~90°
195	06-26	aed9649	U-turn: wheel-heading rotation fallback (IMU offline) + smooth arc ease-in
196	06-27	3126c48	Run SLAM mapping alongside row-follow (--slam), fully decoupled from control
197	06-27	4529eb7	Add per-scan telemetry log (--telemetry) for field analysis / figures
198	06-27	11a16ef	Add optional RL steering (opt-in, sim-trained) + honest pursuit comparison
199	06-27	0614983	RL eval: per-episode CSV export + reward-shaping knobs (accuracy/smoothness)
200	06-27	d14bcc9	RL jerk-penalty sweep: trained policies at c_du 1.0/3.0/8.0
201	06-27	5330be7	Add optional MPC (MPPI) steering with online disturbance observer (--controller mpc)
202	06-27	2fdeef6	SLAM: field coverage map + as-driven trajectory (the operational use case)
203	06-27	31e69ef	Fewer geometry params: drop dead --headland-shift, self-calibrate --row-spacing
204	06-27	d5a397b	U-turn: lower --turn-scrub-comp default 0.6 -> 0.5 (field under-rotation)
205	06-27	a5ac95d	Fix FilterHeading: stop cancelling the FilterState subscription after 5s
206	06-27	d569e7c	Row-spacing self-cal: anchor to the seed (stop 0.52–1.02 m wandering)
207	06-27	0e647f8	U-turn: raise rotation gates for the live-IMU regime (was ending ~30° short)
208	06-27	fd051d9	Add --record: complete reproducible experiment folder per run (for publishing)
209	06-27	5aaafdc	Run record: label the controller/policy in summary + comparison table
210	06-30	2d5bf87	Add scripts/viz_perception.py — poster/paper LiDAR perception figure
211	06-30	0faebdd	viz_perception: add --mode av3d (self-driving-style dense 3D LiDAR render)
212	06-30	ad0dbe7	viz_perception: add --mode av_bev (top-down AV sensor-scope panel)
213	06-30	5d19738	--record: auto-capture a real scan + render the 3 perception figures per run
214	07-01	a0e2206	Fix row-hopping: continuity ("strip-lock") prior in dual-row pairing
215	07-01	4ee9d58	Add raw-scan streaming to --record (--save-scans)
216	07-01	e1da322	Name --record run folders by controller (run_<controller>_<ts>)
217	07-01	3c428e3	Fix row-distance reset on obstacle recovery; add --no-slam
218	07-01	fe8c85f	Render perception figures early (seconds in), not at exit
219	07-01	07b0da9	Fix heading runaway on terrain grade (cross-row weaving / stuck)
220	07-01	fb73acf	Make heading clamp proportional to lateral offset (kill straight-row weave)
221	07-02	5553342	Disable heading consistency clamp by default (restore strip-lock baseline)
222	07-02	6332b84	Add RL drift integrator + field-calibrated sim (cross-slope rejection)
223	07-02	cb0f5b5	Retrain jerk-sweep variants on 5-input arch; regenerate Pareto table
224	07-02	5df46f3	Save scan data from the very first scan; default RL policy
225	07-02	a50bbe2	Re-enable grade heading-runaway clamp; fix figure colormap crash
226	07-02	6536e49	Re-calibrate LiDAR defaults for forward-facing re-mount (yaw 0, tilt 15, height 0.80 m)
227	07-02	3234c94	diag_birdseye: don't crash when matplotlib is missing on the brain
228	07-02	b8543be	Field-verify LiDAR tilt at 21.7 deg for the new forward-facing mount
229	07-02	a8294e6	diag_birdseye: add --mount-height override to test ground-residual fixes
230	07-03	ec7047c	Row-follow: stronger strip-lock, longer-line row-end, refreshed figures
231	07-04	1f6e072	Add diag_filter_imu.py — probe robot pitch/roll source for slope correction
232	07-04	d89968b	diag_filter_imu: reveal OAK IMU accel field + stream live pitch/roll
233	07-04	ade905c	Slope handling: LiDAR-only lateral (roll) ground detrend
234	07-04	aecf40f	Add diag_pitch_rings.py — measure pitch & height from RAW ring geometry
235	07-04	09f6332	Add diag_sensor_model.py — read Velodyne product-ID from the packet
236	07-04	e5fc42d	Fix root cause: sensor is a VLP-16 Puck Hi-Res, not standard VLP-16
237	07-05	1b75214	Propagate Hi-Res / forward-facing calibration across the whole codebase
238	07-05	e003911	Set LiDAR forward x offset to 0.85 m (1 ft ahead of front tires)
239	07-05	033446a	Add render_pov_video.py — cinematic POV video of the LiDAR navigation
240	07-05	d0712f9	render_pov_video: add orbit + split-screen views (add-on)
241	07-05	561cdcc	Advance the baseline controller: leaky integral (disturbance-cancelling) action
242	07-05	d0a396b	CHANGELOG: enhanced baseline controller + POV video tool
243	07-05	23c4eee	Regenerate controller_pareto.csv against the enhanced baseline
244	07-06	cdca450	Residual RL: policy learns a bounded delta on top of the enhanced baseline
245	07-06	3c31ea1	Add trained residual RL policy (jerk 0.3)
246	07-06	9dbdd0f	Expose heading-consistency cap as --heading-cap / --heading-gate
247	07-06	f2fa503	Add trained residual RL policy (jerk 1.0)
248	07-06	ea97a5d	Add trained residual RL policy (jerk 3.0)
249	07-06	8799932	Complete residual RL jerk sweep + regenerate controller Pareto CSV
250	07-06	e858946	Fix ACQUIRE row-end hang: make the escape counter survive end-conf flicker
251	07-06	fdbeb18	Growth-stage-adaptive row fit for tall/dense canopy + slope obstacle detrend
252	07-06	d02ad39	Lock dense-canopy fit against the two remaining PDF failure modes
253	07-06	71ca11e	Intelligent perception layer: self-consistency arbiter + temporal trust filter

#	Date	Commit	Summary
254	07-06	`3747a2e`	Fix headland turn aborting on misaligned clutter (never turned in field)
255	07-06	`67cdf37`	Fix ACQUIRE row-end hang on mid-range clutter (never took the turn)
256	07-07	`a3abadd`	Headland turn: require a STRONG lock to abort EXIT (stop the thrash)
257	07-07	`67db5e7`	Creep-through-gap: creep forward at a lost row instead of guessing patch vs end
258	07-08	`eac70b7`	Stop false mid-row headland + dense-canopy weave at the perception source
259	07-08	`d19966e`	RL: angular slew-rate bound against field left/right over-correction
260	07-08	`727e186`	Dense-vegetation: multi-scan accumulation + offline replay bench
261	07-08	`139d9f9`	Dense-vegetation ②: capture LiDAR intensity for the residue-strip discriminator
262	07-09	`86df129`	Absolute heading ceiling: stop the RL ~90° whip into the crop wall
263	07-09	`700f9e5`	RL field policy: control-effort penalty fixes the "weird turning", keeps tracking edge
264	10-01	`4cc3584`	Onion raised-bed mode: 4-row comb fit, bed-top crop band, row-aligned ROI
265	10-09	`0517e73`	diag_birdseye: --locate-range for far yaw-check targets
266	10-15	`258a5ba`	Bed mode: range-dependent bed floor + straddle offset guard (field fix)
267	10-15	`5deed55`	Bed mode: steer by the raised-bed relief when the crop is invisible
268	10-15	`05c1fba`	viz_perception: legend text colour without labelcolor (old matplotlib on the brain)
269	10-15	`e2ad722`	Bed mode: detect the bed END, never follow grass, no heading ratchet
270	10-15	`2c8fd89`	Onion preset: 3.5 m headland exit (bed-end gate fires ~3 m before the bed end)
271	10-16	`f1209ae`	Onion RL: sim system-ID from the field run, retrained follow_onion policy, onion-tuned MPC
272	10-16	`211858f`	Onion field: smooth driving (no false bed-end stops, steady speed) + serpentine side plan